Closing the Usability Gap in Naked Objects

From Domain Models to Complete Applications

Anonymous Author(s)

Abstract

The **Naked Objects** architectural pattern proposes that complete user interfaces can be derived automatically from domain models. Instead of manually maintaining separate models, component trees, form layouts, validators, persistence mappings, localization files, and AI tool specs, developers declare their domain logic once and allow the runtime engine to project it into every required form.

While early Naked Objects systems successfully demonstrated this derivation, they relied on a narrow grammar—rendering objects as forms and collections as tables. This created a severe "usability gap" that restricted Naked Objects to internal tools and administrative backends.

This paper presents a modern, flexible interaction grammar that closes this usability gap without requiring hand-authored UI component trees. We treat every screen as a **lens** over the model's object graph, and show that most interactions in structured-information software reduce to two core operations on that lens: (1) **moving the lens**—traversing relationships across the object graph via recursive, orientation-flexible master-detail views (Miller columns), and (2) **shaping the lens**—deciding whether each visible child is layered or inlined, and arranging collections along semantic dimensions (such as linear feeds, inline documents, anchored tabs, and aligned matrices).

We demonstrate **Strvct**, an open-source client-side framework implementing this model-driven approach. By centralizing the model-to-view pipeline through slot metadata annotations, Strvct delivers architectural capabilities "for free": automatic responsive collapse, headless testability, transparent persistence and cloud synchronization, unified accessibility, and a natively AI-operable agent projection with a constant-size tool surface.

In a production case study (**undreamedof.ai**, an AI tabletop gaming platform with ~90 domain classes), ~98% of all views are framework-generated. The remaining custom views are inherently graphical media (a 3D dice engine and a battle map), confirming that the boundary of automatic UI generation falls naturally where spatial geometry carries irreducible domain meaning.

CCS Concepts: • **Software and its engineering** → **Object oriented frameworks**; *Model-driven software engineering*; • **Human-centered computing** → *Interactive systems and tools.*

Keywords: naked objects, automatic UI generation, model-driven development, projection, lenses, composable primitives, AI agents, Miller columns

1 Introduction

Modern software development spends immense effort creating and maintaining user interfaces. In traditional component frameworks (such as React, Vue, or Svelte), every screen is authored as a bespoke artifact. When a data model changes—such as adding a field to a customer profile—the developer must manually update:

- The domain model class
- Form UI layouts and input fields
- Client-side and server-side validation rules
- Persistence schemas and ORM mappings
- Navigation graphs and routing states
- Localization string dictionaries
- Accessibility ARIA attributes
- AI agent tool definitions and API endpoints

This structural duplication creates exponential maintenance costs and frequent synchronization bugs.

```
Traditional Component Approach (Manual Propagation):
Domain Model -> UI Components -> Forms -> Validators -> Per

Model-First Naked Objects Approach (Automatic Projection):
                +-> UI Projection (Master-Detail, Tabs, Do
                |                  Accessible & Localized)
                |-> Storage Projection (Local Pools & Clou
Annotated Model -+-> Exchange Projection (JSON Import, Expo
                |-> Agent Projection (Constant-Size AI Too
                +-> Headless Execution & Fast Testing
```

Naked Objects, introduced by Pawson and Matthews in 2002 [11], proposed a radical alternative: expose domain objects directly to users and generate the entire interface automatically. Because there is no separate UI code, consistency is guaranteed—the interface always perfectly reflects the live state of the domain.

Despite its obvious maintenance benefits, early Naked Objects implementations (such as Apache Isis/Causeway [**?**]) struggled with widespread adoption. They derived every object into a form and every collection into a table. While functionally complete, these interfaces felt rigid. They lacked spatial context, viewport adaptability, depth indicators, and visual continuity.

This paper shows how to close that usability gap. We contend that the design space for information management software is remarkably narrow and structured. By replacing generic

forms and tables with a richer grammar—**recursive relationship traversal** and **semantic collection arrangement**—frameworks can derive modern, beautiful, and responsive interfaces directly from annotated models.

The key step is a change of picture. Rather than thinking of an application as a set of screens, we think of it as one complete projection of the model's object graph, and of each screen as a **lens** through which part of that projection is seen (§3).

2 The Usability Gap in Prior Systems

To understand why classic Naked Objects stalled, we must examine the specific usability limitations of early form-and-table systems:

1. **Lack of Spatial Hierarchy:** Generic forms flatten relationships into lists of text boxes and dropdowns. Users cannot visually perceive ownership, containment, or structural depth.
2. **Limited Viewport Adaptability:** Form-based tools are typically designed for fixed desktop resolutions. Adapting them to mobile screens requires ad-hoc, per-screen responsive hacks.
3. **Loss of Context During Navigation:** Drilling into a child object in a form interface usually replaces the screen or pops up a modal window. Each screen becomes a disconnected island, and users never form a picture of the object graph as a whole.

Component frameworks (React, Vue) address this problem by giving developers total visual freedom, but at the cost of authoring every screen by hand. Our goal is to achieve the visual quality and spatial legibility of hand-crafted apps while keeping the zero-UI-maintenance benefits of Naked Objects.

3 The Model as a Graph, the Screen as a Lens

Before describing the grammar itself, it helps to picture what a generated interface is actually showing.

Imagine the domain model as a graph: a tree of ownership, with symbolic links for the references that cross it. Now imagine drawing all of it at once—every object, every property, every collection, fully expanded. This drawing is the model's complete **UI projection**. It is faithful and complete, and for any real application far too large to look at.

Every screen the user sees is a **lens** over that drawing. At first order, a lens makes only two decisions:

1. **Visibility:** Which nodes to collapse. Typically everything off the current navigation path is collapsed, leaving the path itself and the immediate children of the nodes along it.
2. **Placement:** For each visible parent→child edge, whether the child is **layered** (given its own plane beside or in front of its parent, as in Miller columns, push stacks, and tabs) or **inlined** (drawn within its parent's extent, as in documents, grouped sections, and side-by-side panes).

Layering and inlining are not modes of the whole interface but choices made edge by edge, and they compose freely. A layered column may hold an inlined document whose link opens another layer; a document section may itself be a set of anchored tabs. The mixed lens in Figure 1 inlines *Project*'s sections in one column and layers *Task 2* into the next, and even the pure inlining lens layers its link to *Person*. Because the choice lives on each edge rather than on each screen, it can change without touching the model—set by a developer's hint, or switched by the framework as space changes.

At second order, a lens decides **arrangement**: how siblings within a layer or an inlined run are laid out—their orientation, whether their properties align across siblings into a table, or whether a value maps to position, as on a timeline (§4.1).

This framing reduces an apparently unbounded space of screens to something tractable. Any screen is a path through the graph, plus a placement for each visible edge and an arrangement for each visible collection. That is a small, regular decision space—small enough for the framework to own its mechanics (the path, collapse, layering, inlining, and responsive switching) while the developer states intent through hints on the model: which edges inline, which layer, how collections arrange. The decisions are the developer's; their implementation is the framework's.

It also explains behaviors that would otherwise have to be built screen by screen:

- **Navigation** is moving the lens. Traversing a relationship changes the path, not the model.
- **Responsive collapse** is the lens switching edges from inlined to layered as space runs out, within the bounds the hints allow. Miller columns that collapse into a push stack are still layered—in depth rather than side by side. Side-by-side panes that degrade to tabs are the same edge moving from inlined to layered.
- **Links** are followed by layering. Inlining a link would draw its target inside the referrer, and a cycle of links would expand forever; layering keeps every lens finite and makes the crossing visible.

None of these decisions is new to the literature. Furnas's fisheye views [7] formalize visibility as a degree-of-interest function over a tree; overview+detail and focus+context interfaces [3] separate or blend levels of detail much as layering and inlining do; and tree visualization distinguishes children drawn *within* their parents (inclusion) from children drawn *beside* them (adjacency) [12]. Inlining a linked target is what Nelson called transclusion [10]. Systems such as Furnas's, APT [9], and SUPPLE [8] go further and compute the presentation—by interest score, by search, or by

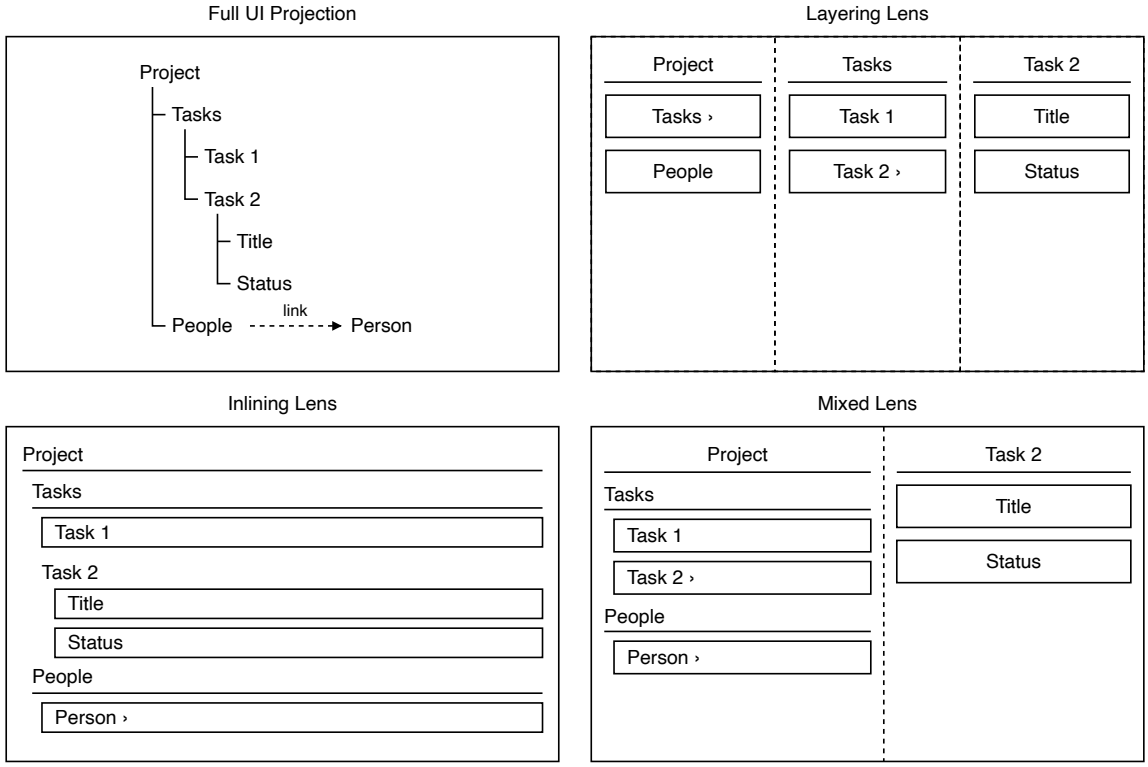


Figure 1. One Projection, Three Lenses: a small model graph as its full UI projection, and through layering, inlining, and mixed lenses.

optimization. Strvct deliberately does not. It owns the mechanism and supplies the defaults; the developer states intent through hints, much as a projectional editor lets a language designer declare the notation through which a model is edited [13].

Our use of *lens* differs from the Magic Lenses of HCI [2], which filter a local region of the screen. It is closer to the lenses of bidirectional programming [6]: editable views whose changes flow back to their source, as edits in a Strvct view flow back to the model.

The UI projection is not the only one. The same graph is projected into records in object pools (the **storage projection**), into JSON for import, export, and patching (the **exchange projection**), and into a schema-and-patch tool interface for AI agents (the **agent projection**, §7.1). Each is a different reading of the same annotated model; none is authored separately.

4 A Narrow Design Space for Information Apps

A common misconception is that auto-generation forces all apps to look like boring administrative dashboards. However, a deeper analysis of consumer and enterprise software reveals a powerful pattern:

> **Most interactions in structured-information applications consist of traversing relationships among domain objects and arranging collections of those objects along semantic dimensions.**

Consider CRMs, project management tools, email clients, social networks, healthcare records, e-commerce stores, and settings panels. Although their visual styling differs, their fundamental operations are identical: users find an object, follow a relationship, inspect or edit details, and organize collections.

```
Typical Domain Graph:
Workspace
+-- Projects
|   +-- Project
|       +-- Tasks
|       +-- People
|       +-- Milestones
+-- Customers
+-- Documents
```

4.1 Relationship Traversal vs. Collection Arrangement

In terms of the lens (§3), we divide the user interface grammar into two distinct layers:

- **Relationship Traversal:** Moving the lens along links in the object graph (e.g., from a *Project* to its *Tasks*, then to a specific *Task*).
- **Collection Arrangement:** Displaying a set of related objects in a specific semantic layout (e.g., viewing *Tasks* as an ordered list, a Kanban board grouped by status, a table, or a timeline).

4.2 The Boundary of Automatic Generation

Where does automatic generation end and custom code begin? The boundary is clean and objective:

- **Generated UI (Domain Model):** Managing structured data where spatial layout is purely an organizational aid (text, relationships, properties, categories, dates).
- **Custom UI (Specialized Media):** Surfaces where spatial position carries *irreducible domain meaning*.

Examples of surfaces requiring custom UI renderers:

- **Creative tools:** Canvas drawing, code editors, rich audio/video sequence editors.
- **Graphical visualizations:** 3D game viewports, interactive geographic maps, medical imaging scans.
- **Data plots:** Complex financial or scientific charts.

For a CRM or settings panel, generated UI covers ~100% of the app. For an IDE or game, generated UI handles the application shell (file trees, properties, settings, inventory), while a specialized custom renderer powers the central canvas.

5 Architecture & UI Mechanics in Strvct

Strvct implements this two-part grammar using client-side JavaScript. Master-detail views are its layering lens (§5.2); inline layouts are its inlining lens (§5.3). The core visual building blocks are **Tiles**, **Tile Stacks**, and **Master-Detail Views**.

5.1 Tiles and Tile Stacks

The fundamental primitive in Strvct is the **Tile**:

- **Summary Tile:** Displays a domain object with a title, subtitle, icon, and contextual actions. Selecting a summary tile opens its details in an adjacent column.

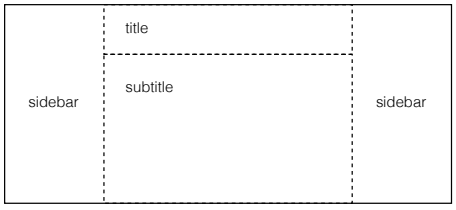


Figure 2. Summary Tile Primitive.

- **Property Tile:** Displays an individual object property (key-value pair) with direct inline editing (text inputs, numbers, toggles, date pickers) and validation feedback.

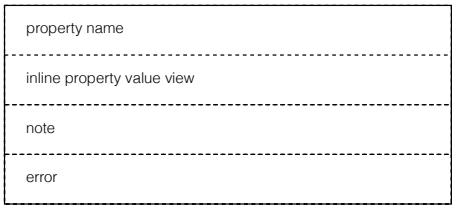


Figure 3. Property Tile Primitive.

A **Tile Stack** is a scrollable, ordered list or grid of tiles. Stacks support drag-and-drop reordering, slide-to-delete, and direct data import/export.

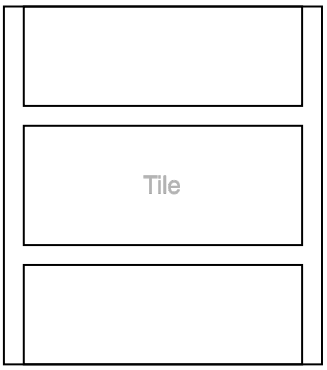


Figure 4. Tile Stack Primitive.

5.2 Recursive Master-Detail Navigation

A **Master-Detail View** pairs a master tile stack with a detail pane showing the selected object's contents. Each selection layers the chosen child in a new pane. Because detail panes can themselves contain child master-detail views, navigation composes recursively to arbitrary depth.

Key characteristics of Strvct's master-detail engine:

- **Flexible Orientation:** Each level can layout horizontally (like macOS Miller column file browsers [4]) or vertically (like nested accordion lists or settings panels).
- **Automatic Mobile Collapse:** When viewport space is constrained, earlier columns automatically collapse into a mobile push/pop stack equipped with interactive breadcrumbs—the same layers, stacked in depth rather than side by side. Zero per-screen responsive design code is required.

Mechanism	Semantic mapping	Familiar UI Presets
Recursive composition	Parent-child relationships or derived groupings	Navigation tree, grouped list, Kanban board
Cross-sibling alignment	Equivalent property slots aligned across items	Data table, spreadsheet, pivot matrix
Value-to-space mapping	Numeric, temporal, or spatial values mapped to position/extent	Timeline, Gantt chart, calendar, map
Quantitative encoding	Numeric values mapped to visual marks or axes	Line chart, bar plot, heatmap

Table 1. Arrangement mechanisms and the familiar UI forms they yield.

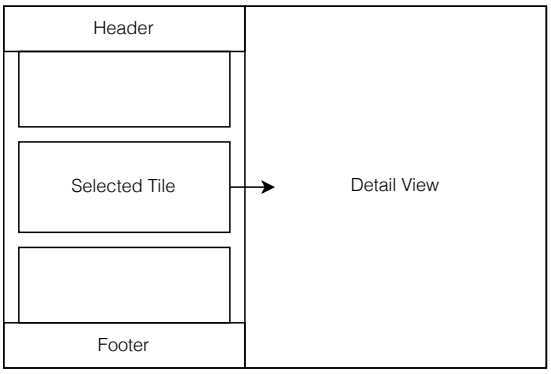


Figure 5. Master-Detail Navigation View.

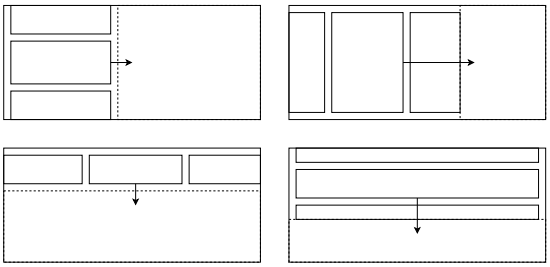


Figure 6. Master-Detail Orientations.

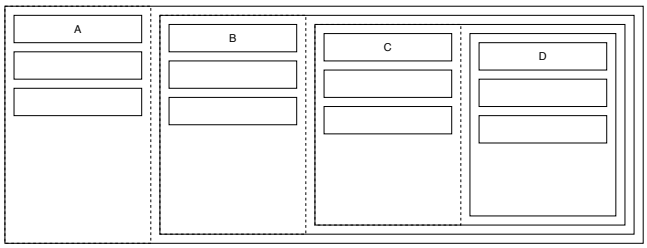


Figure 7. Expanded Viewport (Desktop Layout).

5.3 Advanced Inline Layouts

While column traversal is the default, certain complex screens require rendering child views directly inside parent tiles rather than pushing new columns. These children are inlined rather than layered. Strvct provides three built-in inline placement policies:

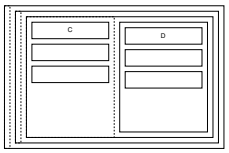


Figure 8. Collapsed Viewport (Mobile Layout with Breadcrumbs).

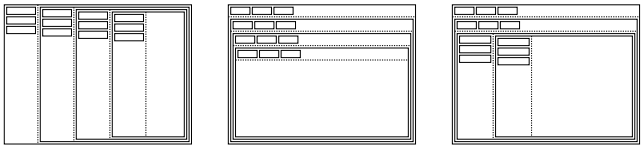


Figure 9. Miller Column Nesting Patterns: Vertical (left), Horizontal (center), Hybrid (right).

1. **Anchored Tabs:** Renders selected children as interactive side-by-side panes or tab bars. Width arbitration automatically fits as many panes as space permits on wide desktop monitors while gracefully degrading to standard tabs on mobile.
2. **Document Inlining:** Renders a nested subtree as a continuous multi-column document with flowing sections, headers, and fields. Fields edit in place without triggering navigation transitions.
3. **Presented Nodes:** Allows an object to delegate its presentation to a derived node (e.g., displaying a structured RPG character sheet instead of raw property slots), providing bespoke reading views without custom component templates.

6 Model Declarations & The Annotation Bridge

Developers define domain models as JavaScript classes inheriting from `SvStorableNode`. Properties are declared as **slots** enriched with metadata annotations:

```
(class Character extends SvStorableNode {

    initPrototypeSlots () {
```

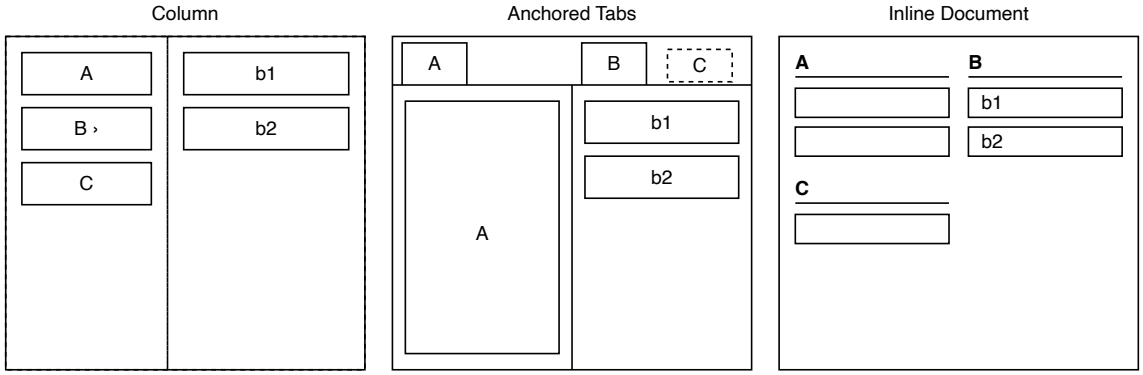


Figure 10. Comparison of Subtree Layouts: Columns, Anchored Tabs, and Inline Document Flow.

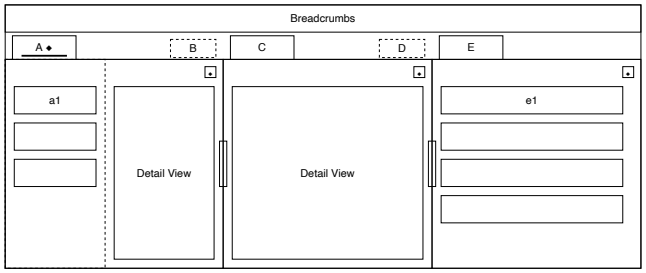


Figure 11. Anchored Tabs (Wide Desktop Viewport).

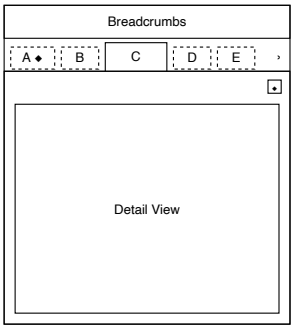


Figure 12. Anchored Tabs (Narrow Mobile Viewport).

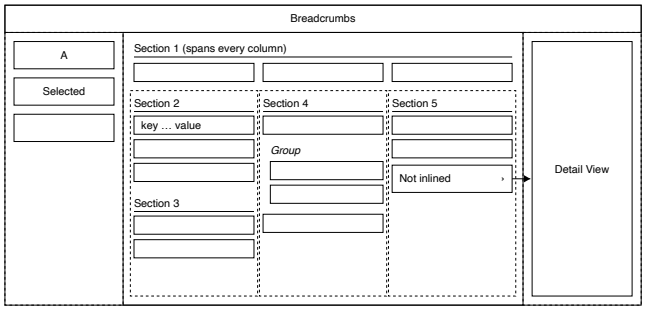


Figure 13. Inline Document Section Flow.

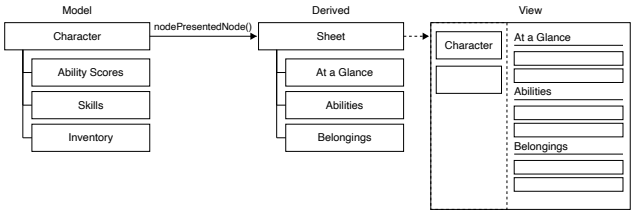


Figure 14. Presented Node Pattern.

```
    {
        const slot = this.newSlot("name", "");
        slot.setSlotType("String");
        slot.setShouldStoreSlot(true);
        slot.setSyncsToView(true);
        slot.setCanEditInspection(true);
    }
    // Level slot: Number property
    {
        const slot = this.newSlot("level", 1);
        slot.setSlotType("Number");
        slot.setShouldStoreSlot(true);
        slot.setSyncsToView(true);
    }
    // Inventory relationship: Navigable child object
    {
        const slot = this.newSlot("inventory", null);
        slot.setFinalInitProto(Inventory);
        slot.setIsSubnodeField(true);
    }
}

subtitle () {
    return "Level " + this.level();
}

// Name slot: String property, persisted, syncs to view, editable
```

6.1 The Multi-Reader Annotation Architecture

Slot annotations serve as a decoupling bridge. Each framework subsystem reads slot metadata independently without cross-system dependencies:

- **UI Layer (UI Projection):** Reads `slotType`, `canEditInspection`, and `isSubnodeField` to instantiate Property Tiles and navigate child subnodes.
- **Persistence Layer (Storage Projection):** Reads `shouldStoreSlot` to serialize records into local storage (IndexedDB) and cloud object pools.
- **Sync Scheduler:** Reads `syncsToView` to observe slot updates and post coalesced, deferred notifications to active views.
- **Agent Projection:** Reads property types and constraints to auto-generate JSON validation schemas and patch tools.
- **Localization Engine:** Reads property names and translation contexts to auto-translate labels.

```
                +-> UI Rendering Layer (Property Tiles & Controls)
                |-> Persistence Engine (IndexedDB & Cloud Commits)
Slot Metadata ----+-> Sync Scheduler (Weak Observer Notifications)
                |-> AI Agent Infrastructure (Patch Schemas & Verification)
                +-> Internationalization Engine (Label Translation)
```

7 Structural Consequences ("Free" Capabilities)

Centralizing the model-to-view pipeline through slot annotations yields powerful architectural capabilities that traditional component frameworks must build and maintain manually for every screen:

7.1 Natively AI-Operable Model

Because slot metadata describes every class, Strvct exposes a universal agent projection: a tool surface consisting of just two generic functions:

1. `schema-fetch`: Retrieves the slot schema for any domain class or object instance.
2. `apply-patch`: Applies a JSON patch payload to update properties or mutate child relationships.

This tool surface remains **constant in size** regardless of whether the application contains 5 domain classes or 500. When an AI agent generates an invalid field patch, the rejected response includes the slot's schema, allowing the model to self-correct in real time.

7.2 Headless Execution and Ultra-Fast Testing

Domain nodes contain zero DOM or browser dependencies. The exact same domain classes run unchanged under Node.js. Unit and integration tests instantiate model graphs, invoke domain methods, and assert against slot states at lightning speed without spinning up headless browser instances (Puppeteer/Playwright).

7.3 Transparent Persistence and Cloud Sync

Strvct divides persistence into two unified strategies:

- **Local Object Pools:** Fast, synchronous IndexedDB storage for domain object state.
- **Content-Addressable Blob Store:** Asynchronous SHA-256 keyed storage for large binary assets (images, audio, files).

Cloud synchronization sends versioned atomic commits containing only the modified slot records. Conflicts are detected deterministically per document pool.

7.4 Universal Accessibility and Localization

Accessibility and localization are properties of the UI projection rather than projections of their own, and both are handled at the framework primitive level: translation is applied at the model-to-view boundary, as labels leave the model. Because every view is rendered by ~15 shared view classes, ARIA roles, focus management, keyboard navigation, and translation hooks are paid once. Fixing an accessibility or translation bug in a primitive tile fixes it across every screen in the entire application.

7.5 Live Inspector & Developer Mode

Every tile supports modifier-clicking (Option + Click) to reveal a live Property Inspector displaying all declared slots, storage flags, and validation states. Developers can toggle Developer Mode to inspect hidden internal subnodes without writing custom debugging views.

8 Empirical Case Study: undreamedof.ai

Strvct was used to build **undreamedof.ai**, a commercial AI-powered virtual tabletop application for tabletop roleplaying games (D&D 5e).

Subsystem	Domain classes	Custom views
Character system	~30	0
Campaign hierarchy	~20	1 (Map Viewer)
Session & Gameplay	~25	1 (3D Dice Engine)
AI Integration & Tools	~15	0
Total	**~90**	**2**

Table 2. undreamedof.ai breakdown by subsystem.

Over 98% of all application screens—character sheets, inventory managers, spellbooks, campaign trees, settings panels, chat logs, and admin inspectors—are derived automatically by Strvct.

The only two custom views in the entire application are:

1. A **3D WebGL Dice Roller**
2. An **Interactive Tactical Battle Map**

Both represent specialized graphical media where spatial geometry carries irreducible domain meaning (§4.2). Their

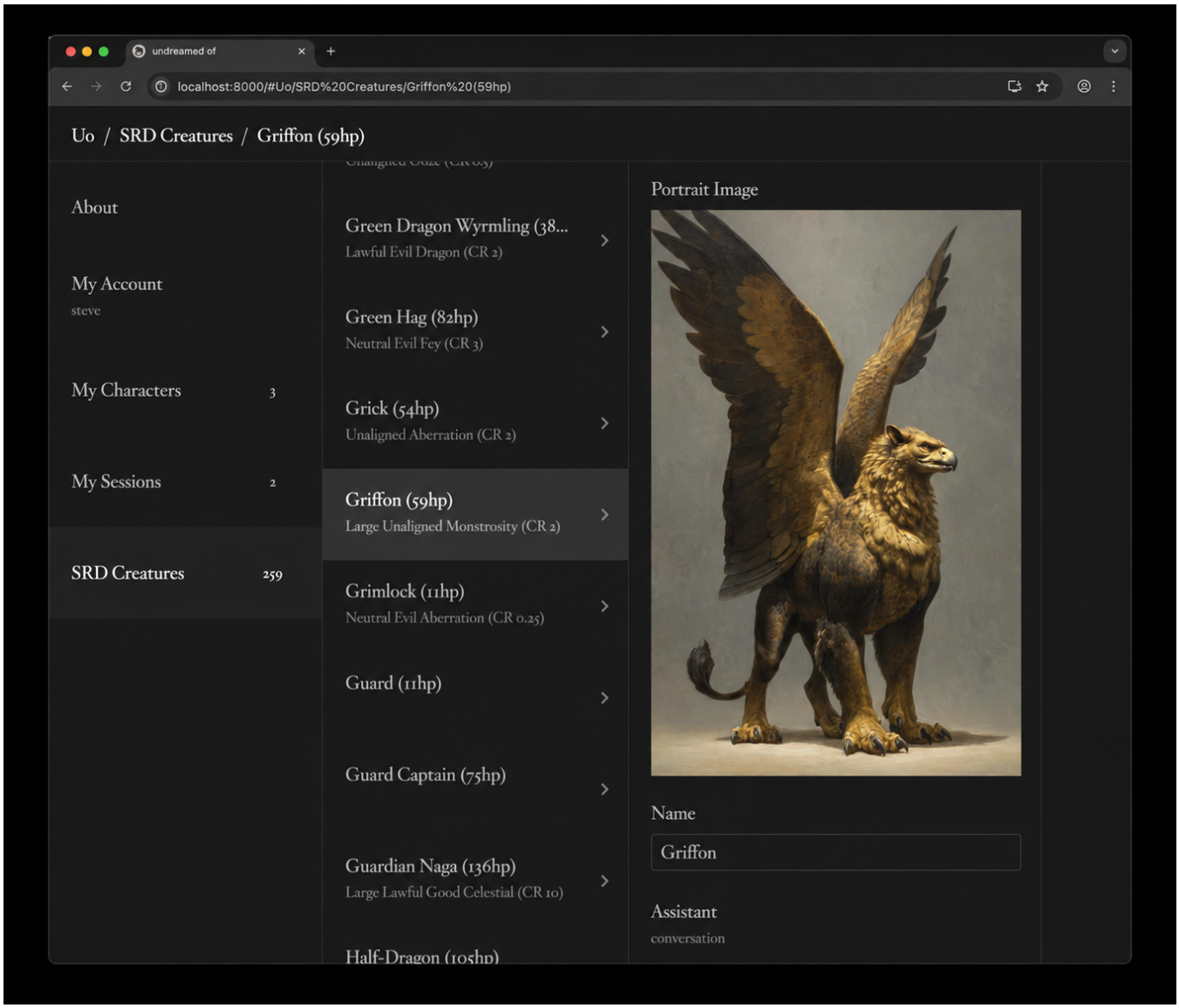


Figure 15. undreamedof.ai built with Strvct.

presence confirms that Strvct's boundary between auto-generated and custom UI falls exactly where predicted.

9 Comparison with Related Paradigms

10 Limitations & Future Work

While Strvct successfully demonstrates the power of modern Naked Objects, several open challenges and limitations remain:

1. **Inherently Graphical Media:** Applications centered on canvas editing, CAD, video sequencing, or 3D gaming will always require custom view renderers.
2. **Ecosystem Isolation:** To support content-addressable bundling and zero-build client loading, Strvct uses a custom resource loader rather than standard npm/Webpack/Vite tooling.
3. **Aligned Grid Arrangements (Tables & Spreadsheets):** While inline document flow and anchored tabs are production-proven, cross-sibling column alignment (making row tiles align into full spreadsheets) is currently a proposed framework extension.
4. **Extreme Collection Scaling:** Lazy view instantiation keeps rendering lightweight, but virtualized viewport scrolling for mega-collections (10^5+ items) remains an active area of optimization.
5. **User-Chosen Lenses:** Because placement and arrangement are per-edge hints rather than authored screens,

Dimension	Traditional Component Frameworks	Low-Code / Visual Builders	Strvct (Modern Naked Objects)
Primary Artifact	Authored Component Trees	Visual Layout Diagrams	Annotated Domain Model
UI Derivation	Manual per screen	Manual per screen	100% Derived automatically
Model/UI Coupling	High (manual props & state)	High (visual bindings)	Zero (model has no UI code)
Maintenance Cost	Scales with screen count	Scales with screen count	Near constant (model-driven)
AI Agent Interop	Per-screen tool specs	Custom API connectors	Native constant-size tool surface
Offline & Cloud Sync	Complex manual sync logic	Server-dependent	Built-in local pools & versioned commits

Table 3. Comparison of UI Development Paradigms.

users could in principle reshape their own lenses—inlining a section they always want open, or layering one they rarely need. This is possible today through the live inspectors (§7.5), but not every hint is persisted; a supported, persistent user-facing control is future work. Engelbart's NLS gave users *viewspecs*—how many levels to show, how many lines per item—for much the same purpose [5].

11 Conclusion

For twenty-five years, the Naked Objects pattern offered an enticing promise: write the domain model, and the application will follow. Early implementations stalled not because derivation was impossible, but because forms and tables were not general or flexible enough to deliver modern user experiences.

Strvct solves this by establishing a complete, flexible interaction grammar based on **recursive relationship traversal** and **semantic collection arrangement**. By eliminating manual UI component authoring, developers eliminate view-layer maintenance, reduce synchronization bugs, and gain native AI operability, instant headless testing, transparent persistence, and universal accessibility.

As AI agents become routine co-processors of software state, the cost of maintaining separate models, UI components, and tool specs will become unsustainable. Model-first derived architectures offer a durable, elegant alternative: software where the user interface is a live, faithful projection of the domain model.

References

[1]]causeway Apache Software Foundation. [n. d.]. Apache Causeway. https://causeway.apache.org/ Formerly Apache Isis.

[2] Eric A. Bier, Maureen C. Stone, Ken Pier, William Buxton, and Tony D. DeRose. 1993. Toolglass and Magic Lenses: The See-Through Interface. In *Proceedings of SIGGRAPH '93*.

[3] Andy Cockburn, Amy Karlson, and Benjamin B. Bederson. 2008. A Review of Overview+Detail, Zooming, and Focus+Context Interfaces. *Comput. Surveys* 41, 1 (2008), 1–31.

[4] Wikipedia contributors. 2024. Miller columns. https://en.wikipedia.org/wiki/Miller_columns

[5] Douglas C. Engelbart and William K. English. 1968. A Research Center for Augmenting Human Intellect. In *Proceedings of the AFIPS Fall Joint Computer Conference*.

[6] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Transactions on Programming Languages and Systems* 29, 3 (2007).

[7] George W. Furnas. 1986. Generalized Fisheye Views. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '86)*. 16–23.

[8] Krzysztof Gajos and Daniel S. Weld. 2004. SUPPLE: Automatically Generating User Interfaces. In *Proceedings of the 9th International Conference on Intelligent User Interfaces (IUI '04)*. ACM.

[9] Jock Mackinlay. 1986. Automating the Design of Graphical Presentations of Relational Information. *ACM Transactions on Graphics* 5, 2 (1986), 110–141.

[10] Theodor H. Nelson. 1981. *Literary Machines*. Mindful Press.

[11] Richard Pawson and Robert Matthews. 2002. *Naked Objects*. John Wiley & Sons.

[12] Hans-Jörg Schulz, Steffen Hadlak, and Heidrun Schumann. 2011. The Design Space of Implicit Hierarchy Visualization: A Survey. *IEEE Transactions on Visualization and Computer Graphics* 17, 4 (2011), 393–411.

[13] Markus Voelter, Janet Siegmund, Thorsten Berger, and Bernd Kolb. 2014. Towards User-Friendly Projectional Editors. In *Software Language Engineering (SLE 2014)*. 41–61.